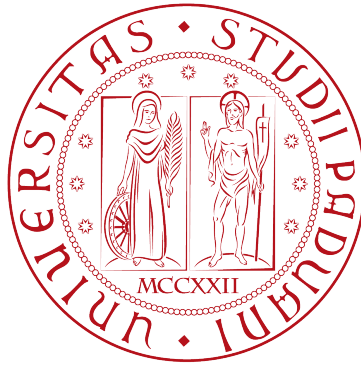




UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

MASTER DEGREE IN ICT FOR INTERNET AND MULTIMEDIA

**Cybersickness detection in virtual reality
experiences: a neural network approach**

SUPERVISOR:

DR SARA BALDONI

CO-SUPERVISOR:

MATTEO CALIGIURI

CANDIDATE:

LUCA GIURIATO

STUDENT ID: 2120924

ACADEMIC YEAR 2024-2025

Abstract

Cybersickness is a common problem many users face during virtual reality experiences. The causes of this phenomenon are still not certain, and different theories have been developed to give an explanation for this phenomenon and are used as a starting point for the different solutions studied in recent years to automatically detect it and possibly prevent it. In this thesis, a Deep Learning approach based on the use of neural networks is proposed. The dataset used comes from a subjective test involving 60 subjects during which information about the 6 degrees of freedom of the VR headset was recorded. For each degree of freedom, the related information is stored in a heatmap, an image that represents its temporal evolution. Each set of heatmaps is paired with a label to discriminate between absence and presence of cybersickness, based on a state-of-the-art Simulation Sickness Questionnaire participants were asked to compile. These heatmaps are used to train an on-purpose defined neural network composed of two encoders that take the heatmaps as input. The encoded data produced by the two encoders is then combined to perform a binary classification to detect the presence of cybersickness in the analyzed subject, labeling the heatmaps and later comparing the newly created labels with the original labels. The model achieved an accuracy of 97.5%, which is proving that it could represent a solid foundation for the development of future cybersickness detection models.

Sommario

La *cybersickness* è un problema comune di cui molti utenti soffrono durante esperienze di realtà virtuale. Le cause di questo fenomeno non sono ancora state accertate e sono state formulate diverse teorie a riguardo, usate successivamente come punto di partenza da diversi studi per riconoscere automaticamente e prevenire il fenomeno. Questa tesi presenta un approccio di Deep Learning basato sull'uso di reti neurali. Il dataset utilizzato deriva da dei test soggettivi a cui hanno preso parte 60 partecipanti, durante i quali il visore ha registrato i dati riguardanti i 6 gradi di libertà delle loro teste. Le informazioni riguardanti i diversi gradi di libertà della testa sono state impresse in delle *heatmaps*, delle immagini che ne rappresentano l'evoluzione nel tempo. Ogni gruppo di *heatmaps* è stato associato a una *label* che attesta la presenza o l'assenza di *cybersickness* tramite un questionario di *Simulation Sickness* conforme allo stato dell'arte e compilato dai partecipanti stessi. Queste *heatmaps* sono state usate per allenare una rete neurale ideata per il riconoscimento automatico della *cybersickness*. La rete neurale è composta da due *encoders* che prendono le *heatmaps* in ingresso. I prodotti dei due *encoders* vengono successivamente concatenati per svolgere una classificazione che stabilisce se le *heatmaps* in ingresso siano associate o meno alla presenza di *cybersickness*. Questa classificazione viene poi confrontata con le *label* corrispondente. Il modello ha raggiunto una accuratezza del 97.5%, provando la sua validità come possibile base da cui sviluppare futuri modelli per individuare la *cybersickness*.

Acknowledgements

A special thanks goes to Dr. Sara Baldoni and PHD Matteo Caligiuri for accepting me to participate in their project, closely and kindly guiding me through each step in the process of writing this thesis. They have been the best mentors I could ever imagine.

Contents

Abstract	III
Sommario	III
Acknowledgements	VII
1 Introduction	1
2 Related Works	3
2.1 Factors that contribute to Cybersickness	3
2.2 Causes of Cybersickness	4
2.3 Prevention and Reduction of Cybersickness	6
2.4 Cybersickness detection techniques	10
3 Proposed Method	13
3.1 The dataset	13
3.2 Proposed model	18
3.3 Training and testing the model	20
4 Experimental Results	25
5 Conclusions	33
Acronyms	35
Bibliography	36

List of Figures

2.1	Representation of the vestibular system. One of the function of this system is to gather information about the motion of your body. The utricle senses linear acceleration of the body and the semi-circular canals sense angular acceleration of the body [1].	5
2.2	A frame of a Virtual Reality (VR) application showcasing Field of View (FOV) reduction and peripheral blurring. The upper section of the frame has not been modified, peripheral blurring is applied to the middle section and FOV reduction is applied to the lower section [2].	7
2.3	Four frames from a VR application. The left column contains the unmodified frames. On the right column, gaze-contingent image deformation is applied to reduce vection [3].	8
2.4	Sequences of frames of a rotation in a VR environment. The upper sequence belong to a normal rotation in the environment. The lower sequence is the result of the application of rotation snapping to the same rotation that is shown in the upper sequence [4].	9
2.5	Sequences of frames of a translation in a VR environment. The upper sequence belong to a normal translation in the environment. The lower sequence is the result of the application of translation snapping to the same translation that is shown in the upper sequence [4].	9
2.6	Application of rotational blurring to the frames of a first-person shooting game. The upper sequence is unmodified. The lower sequence is the result of the application of rotational blurring to the upper sequence [5].	9
2.7	On the left, Galvanic Skin Response (GSR) sensor attached to index and middle finger and Head Mounted Display (HMD). On the right, Zephyr OmniSense BioHarness-5.0 sensor to acquire Electrocardiogram (ECG) [6].	10

3.1	Example of position heatmaps belonging to the same subjective test. Each heatmap represent the information regarding translation in a specific axis. . .	14
3.2	Illustration of the three degrees of freedom of rotation of the human head [7].	15
3.3	Example of orientation heatmaps belonging to the same subjective test. Each heatmap represent the information regarding rotation in a specific orientation.	16
3.4	The six heatmaps merged into two Red, Green and Blue Images (RGB) images, cut to 10 minutes and normalized.	17
3.5	Illustration of the autoencoder structure [8].	18
3.6	Illustration of a Convolutional Neural Network (CNN). Convolutional layers, followed by a flatten layer and the classification layers [9]	19
3.7	Diagram of our model. Position and orientation heatmaps are encoded and the resulting feature are concatenated for classification. The final output will be a label signaling either absence or presence of Cybersickness (CS).	20
3.8	Diagram of our model. Position and orientation heatmaps are encoded and the resulting feature are concatenated for classification. The final output will be a label signaling either absence or presence of CS.	21
3.9	An example of confusion matrix. The two rows correspond to the true labels of the samples and the two columns correspond to the labels predicted by the model [10].	22
4.1	Results of training and testing the one-encoder model with only position heatmaps from the original dataset.	26
4.2	Results of training the one-encoder model with a dataset composed of 33% “present” samples.	27
4.3	Results of testing the one-encoder model trained with a dataset composed of 33% “present” samples.	28
4.4	Results of training the one-encoder model with a dataset composed of 50% “present” samples.	29
4.5	Results of testing the one-encoder model trained with a dataset composed of 50% “present” samples.	30
4.6	Results of training and testing the larger model with the number of features from each encoder set to 192.	31
4.7	Results of training and testing the larger model with the number of features from each encoder set to 5.	32

Chapter 1

Introduction

VR has been used for multiple purposes in different fields, not only as pure entertainment, but also as a valuable tool of simulation in many professional training procedures and in project development in many different fields. Examples of VR applications can be seen in flight simulation for future pilots, in the creation of new design for architectural and engineering projects, in the healthcare system, and many more.

One of the main issues keeping VR technologies from spreading is CS, a term that describes the state of an individual manifesting a series of symptoms while immersed into a VR experience. Some of the possible symptoms of CS are the eye strain, headache, pallor, full stomach, disorientation, nausea and vomiting [1]. The symptoms are similar to the ones of motion sickness. These similarities have led many researchers to find an explanation for CS in previously formulated theories that address the causes of motion sickness, such as the sensory conflict theory, the poison theory, and the postural instability theory.

Studies have also been conducted to find efficient techniques to reduce the presence and intensity of the symptoms of CS. For instance, the VR screen content can be modified to reduce visual cues that have been identified as causing factors of CS. On the other hand, these modifications can negatively impact the users who do not experience any symptom of CS, lowering their sense of presence, and disrupting their overall experience while immersed in the VR scene. Adopting these techniques only when CS is actually occurring is beneficial to avoid those drawbacks when the user is not experiencing any symptom of CS, but requires some form of CS detection.

This direction has not been fully explored yet, with many recent studies presenting very different approaches for the implementation of their own CS detection mechanism.

For this reason, this thesis presents a new approach for CS detection based on deep learn-

ing. The proposed model is trained with a dataset of on-purpose defined heatmaps, representing the information about the 6 degrees of freedom of the head. The heatmaps were produced with an existing dataset consisting of head tracking data collected during subjective tests. The tests involved 60 participants who experienced four different games: “Beat Saber” (a game in which the gamer has to hit cubic boxes with a sword following the music), “Medal of Honor: Above and Beyond” (a shooting game in the context of the Second World War), “Cooking Simulator” (a simulation of a cooking experience in VR), and “Forklift Simulator” (a simulation of a counterbalance factory forklift in VR).

The proposed model makes use of encoders to extract features from the heatmaps and uses the extracted features to detect which participant experienced symptoms of CS. Two different versions of the model are presented: a lighter model for CS detection with one encoder that only requires partial information (either the 3 degrees of rotation of the head or the 3 degrees of translation of the head), and a more complex model with two encoders that detects CS from full information about all 6 degrees of freedom of the head.

After this introductory chapter, Chapter 2 reviews the literature regarding the most accredited theories explaining the causes of CS and the related contributing factors, different methods to prevent the presence or to reduce the severity of symptoms of CS, and the state of the art of Deep Learning (DL) approaches in CS detection. Chapter 3 presents the proposed method. It discusses the dataset features and its adaptation for the detection task, the structure of the encoders, and the metrics and tools used to evaluate the performance of the model. Chapter 4 presents and discusses all the experimental results, from the first trials with the lighter model to the final results with the more complex model. Finally, Chapter 5 summarizes the results obtained and discusses future improvements and possibilities.

Chapter 2

Related Works

2.1 Factors that contribute to Cybersickness

The factors that contribute to the presence of CS in VR experiences can be divided into three main categories, which are contextual contributors, individual differences and technical system [11]. Contextual contributors is a category that describes all the factors contributing to CS that come from the context in which a VR HMD is used. All environmental factors contributing to CS belong to this category (temperature and humidity are examples of environmental factors). This category also includes the performance requirements demanded by the VR application: the more the activity is demanding, the more the individual will be put under stress. Habituation and exposure time to the VR activity also belong to this category, since habituation can decrease the symptoms of CS and since CS symptoms increase proportionally to exposure time [11]. Individual differences is a category describing all factor contributing to CS that are characteristic of the individual experiencing VR. All traits of the human body contribute to this category, such as Body Mass Index (BMI), general health condition, propensity to migraines, sight problems, and more. In addition to that, the behavioral condition of the individual also has some influence on CS, for example how much time has passed since the last meal or the individual sleep pattern. This category also includes the propensity for CS, considering previous negative experiences with VR and history of motion sickness. Psychological traits also contribute to CS, for example the degree of risk-taking and motivation [11]. With regard to gender and age, these two factors are still very debated, since multiple studies have shown very different results. Female users seem to be slightly more affected than male users. On the other hand, difference in CS presence and intensity appears to be much more related to individual characteristics for female users than for male users. Age shows opposite tendencies

in recent studies instead. Dilanchian *et al.* [12] support an inverse relationship between presence and CS, and found that older people seem to experience a greater presence during VR experiences, with consequently lower CS. On the other hand, Petri *et al.* [13] reported that the *over-60* group on their subjective test experienced significantly more intense symptoms of CS. The technical system category identifies all the aspects of the software and hardware used that can induce or increase motion sickness. For example, high latency between the real motion of the user and the visual motion in the VR and a low frame rate are contributing factors of CS since they disrupt the sense of presence of the users. Other factors included in this category are unrealistic motion (for example, going up some stairs in VR conflicts with information of the vestibular system), flicker, rapid changes in acceleration and wide field of view [14].

2.2 Causes of Cybersickness

To this day, the causes of CS are not completely clear, but there are many different theories that try to explain what the causes of CS are. Between all of them, three different theories are the most supported: the sensory conflict theory, the poison theory and the postural instability theory [1]. Sensory conflict theory was initially developed to explain motion sickness [15] and it relates CS to discrepancies between the information collected by the vestibular system and the information collected through vision. A representation of the vestibular system is provided in Figure 2.1. Part of the vestibular system, which has the primary function of hearing sounds through our ears, is also responsible for collecting information regarding angular and linear acceleration [1]. In certain situations, a subject that is resting in a stationary position can perceive a sense of self-motion, for example sitting in a train at the station waiting for its departure, while another train passes by it is possible to feel like your train is moving in the opposite direction. This perception of self-motion generated by visual stimuli is calledvection and can also occur due to optical flows generated by the display of a HMD. Vection does not come with the same vestibular information that would be present in real cases of self-motion, information that is learned and reinforced through experience. This theory identifies this discrepancy as the primary cause of CS but it does not explain exactly why a person gets sick. It also lacks predictive power for both presence and severity of the symptoms of CS and it does not account for all factors mentioned in the previous section, proven to be associated with CS [1]. Poison theory looks at motion sickness and CS from an evolutionary perspective [16]. When an individual ingests some poison, the coordination of various input systems, including vestibular and visual ones, is compromised by the physiological effects of the poison. These

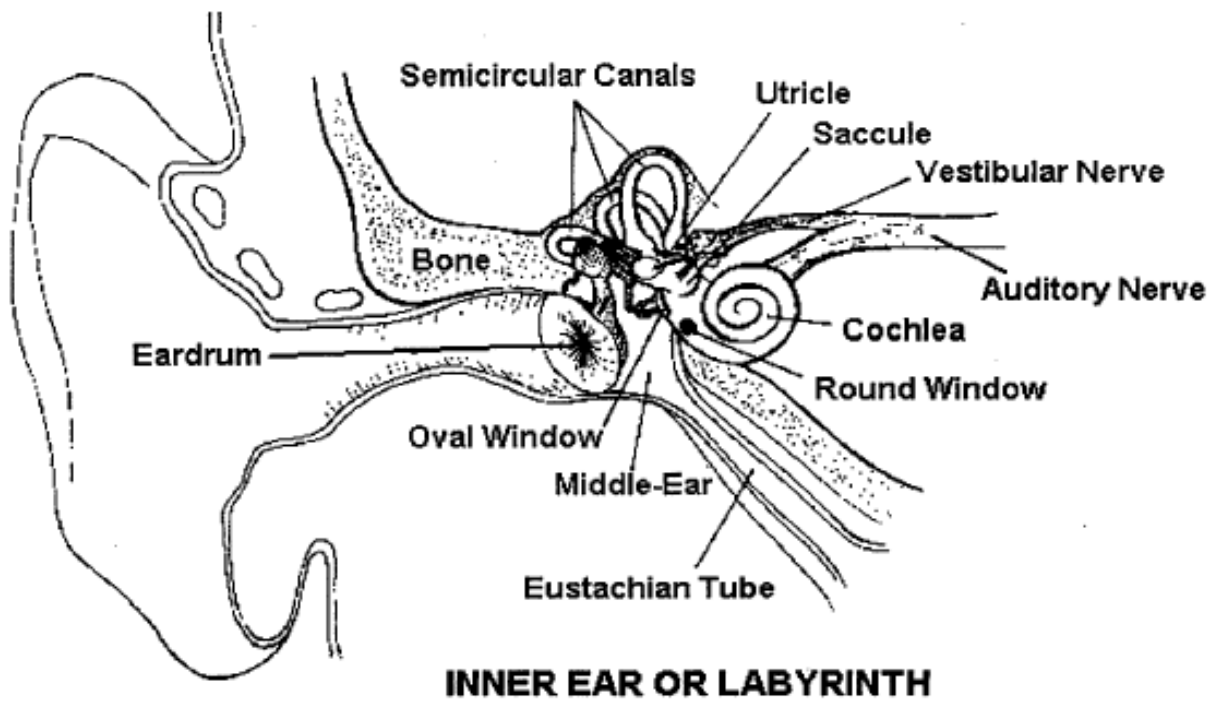


Figure 2.1: Representation of the vestibular system. One of the function of this system is to gather information about the motion of your body. The utricle senses linear acceleration of the body and the semi-circular canals sense angular acceleration of the body [1].

early symptoms, which act as a warning signal, let the individual be aware of the danger, and, starting from an evolutionary standpoint, the individual will increase his chances of surviving by removing the poison from his stomach. This theory relates this scenario to VR experiences stating that the intense stimuli coming from the VR simulation are affecting our input systems in a very similar way to that of a poison, causing all the symptoms associated with CS [1]. This theory has been supported by research [17], but it does not explain why some individuals experience CS without feeling nausea and lacks predictive power on presence and intensity [1]. Postural stability is defined as “the state in which uncontrolled movements of the perception and action systems are minimised” [18]. The sources of postural instability are environmental and individuals learn how to react to changes in the environment in order to maintain postural stability through experience. As an example, the first time a person walks on ice usually falls until he or she understands how to change the walking pattern to be stable and not fall [1]. You do not need to fall in order to be in a state of postural instability, instability is present even when the control of the body is only degraded and not fully lost. Between the categories of situations that can cause postural instability, VR experiences fall into the category of altered specificity [18]. This means that, during VR experiences, postural stability can be lost due to optically specified accelerations that are not present in physical space, leading people to specific reactions and influencing the strategies the individual adopts to maintain

postural stability. A prolonged postural instability causes CS of an intensity proportional to the degree of the instability. This theory was proposed as an attempt to refute the sensory conflict theory since there are cases where the absence of redundancy between vestibular and visual information does not induce CS [1].

2.3 Prevention and Reduction of Cybersickness

Multiple researchers have tried new approaches to address symptoms of CS during VR experiences. One of the approaches taken was to stimulate the vestibular system as a remedy for the absence of vestibular information whenvection occurs. Weech *et al.* [19] used Galvanic Vestibular Stimulation (GVS) to prevent CS from happening. GVS is a procedure in which the semi-circular canals (perception of angular acceleration [1]), the otholit organs (perception of linear acceleration [1]) and adjacent vestibular nerves are activated by a very small electrical current (usually between 1 and 2 mA) induced by opposite charged poles [20], imperceptible, applied behind the ears. Weech *et al.* [19] used both a noisy alternate current (1,750 mA of amplitude) and a sham group with zero current. They used both intense and moderate VR activity. They found that, during intense activity, GVS subjects reported lower symptoms during stimulation. Unfortunately, this difference disappeared in just between 3 and 6 minutes if GVS is removed during activity. On the contrary, GVS and sham group had no perceivable difference in CS during moderate activities. Benelli *et al.* [21] used transcranial alternating current stimulation at 10 Hz to reach the vestibular cortex (“...portion of the cerebrum which responds to input from the vestibular system...” [22]) bilaterally. This method reduced CS in 67% of the subjects, but it works only as a countermeasure since it does not have an impact on recovery time from CS residual symptoms. Numerous researches tried to find techniques that could be applied directly in the VR software. Fernandes *et al.* [23] explored the possibility of reducing the FOV to combat CS, since it seemed reducing FOV lowers symptoms of CS at the cost of a lower presence. They implemented dynamic FOV that changes slightly and automatically in response to visually perceived motion during the virtual experience. During subjective tests, this method increased the time participants stayed inside VR and also increased their level of comfort. Most of the participants did not notice the reduction in FOV and the few that did preferred it to the standard FOV. The study concluded that the best result is obtained with a FOV of 80° where the best trade-off between reduction of CS and the loss of presence is met. Groth *et al.* [2] proposed a similar FOV reduction method and compared it with peripheral blurring. Both methods are shown in Figure 2.2. Both techniques aim to reduce CS without



Figure 2.2: A frame of a VR application showcasing FOV reduction and peripheral blurring. The upper section of the frame has not been modified, peripheral blurring is applied to the middle section and FOV reduction is applied to the lower section [2].

being obtrusive. FOV reduction had a better impact on CS compared to peripheral blurring, strengthening the results from the study previously discussed with an optimal FOV of 80° . Another method that acts on the peripheral portion of the images was still proposed by Groth *et al.* [3] involving gaze-contingent image deformation. In this method, peripheral areas of the frames are warped following the estimated optical flow, aiming to reduce visual vection in an area where this kind of change is not noticed by the user. Examples of warped frames are shown in Figure 2.3 This study found the warping reduced CS scores by 31.8% and halved the dropout rate during the test. 84% of subjects preferred the warped experience even without acknowledging the difference. Farmani *et al.* [4] proposed a different approach, evaluating discrete viewpoint control to reduce CS. In their study, they developed two techniques, rotation snapping and translation snapping. Rotation snapping consists of the discretization of the rotation of the viewpoint. They found an optimal threshold for the rotation speed. In order to reduce CS symptoms, an application using a continuous rotation speed greater than the threshold should adopt rotation snapping instead. The viewpoint is not changed in a continuous way, but it “snaps” from the starting viewpoint to the rotated one without the intermediate steps. Translation snapping is a similar technique, but applied to translation, using short jumps from the starting position to the translated one instead of continuously moving in the desired direction. Example of both techniques are shown in Figures 2.4 and 2.5. The participants stated that at first these techniques are disorienting, but just after a couple of minutes they got used to them and could control their actions much better. Rotation snapping and translation snapping reduced CS by 40% and 47%, respectively. They show great potential, but not all applications are suitable for their use. For example, a very dynamic first-person shooting

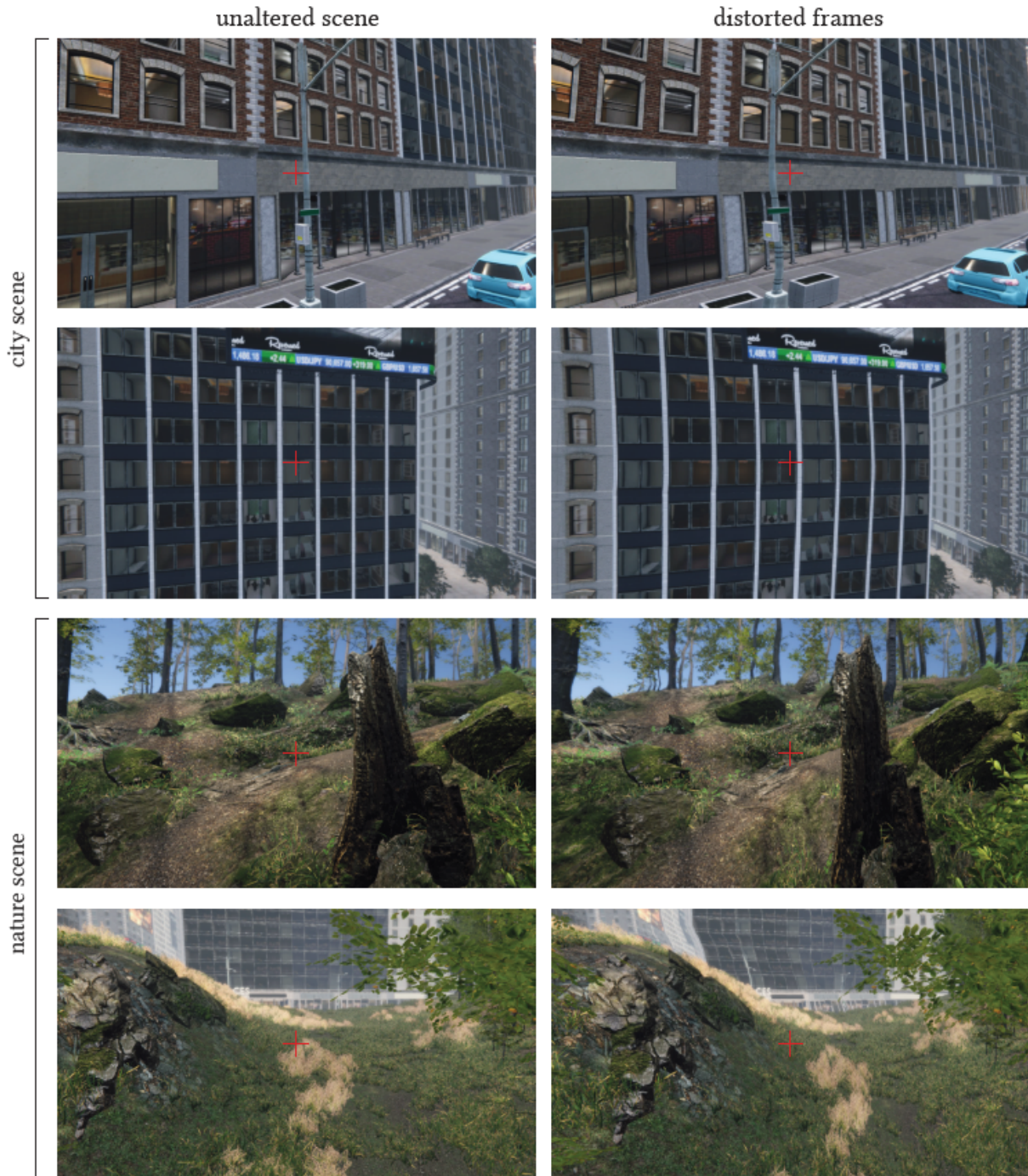


Figure 2.3: Four frames from a VR application. The left column contains the unmodified frames. On the right column, gaze-contingent image deformation is applied to reduce vection [3].



Figure 2.4: Sequences of frames of a rotation in a VR environment. The upper sequence belong to a normal rotation in the environment. The lower sequence is the result of the application of rotation snapping to the same rotation that is shown in the upper sequence [4].



Figure 2.5: Sequences of frames of a translation in a VR environment. The upper sequence belong to a normal translation in the environment. The lower sequence is the result of the application of translation snapping to the same translation that is shown in the upper sequence [4].

game requires fluid and continuous motion, which consequently requires a different strategy to combat symptoms of CS. For this kind of application, Budhiraja *et al.* [5] studied the effects of rotational blurring, which consists of applying uniform blurring during rotation movements to reduce visually induced vection. The application of rotational blurring is shown in Figure 2.6. Not all participants in this study benefited from rotational blurring, some of them did not experience any difference, and some were negatively affected. On the other hand, participants who suffered acute CS had their mean CS level fall substantially, indicating rotational blurring



Figure 2.6: Application of rotational blurring to the frames of a first-person shooting game. The upper sequence is unmodified. The lower sequence is the result of the application of rotational blurring to the upper sequence [5].

could be very effective for this kind of user.

2.4 Cybersickness detection techniques

Multiple studies have proposed Machine Learning (ML) and DL approaches to detect CS in VR experiences starting from data coming from subjective tests. Islam *et al.* [6] proposed CS detection and prediction using deep neural networks and user's physiological data. They tracked GSR data (stress-induced change in the electrical resistance of the skin) applying a GSR sensor on the index and middle fingers, and they obtained Heart Rate (HR) and Breath Rate (BR) from an ECG acquired with a Zephyr OmniSense BioHarness-5.0 sensor. The application of these sensors is shown in Figure 2.7. Their neural network is based on a CNN-LSTM model that takes



Figure 2.7: On the left, GSR sensor attached to index and middle finger and HMD. On the right, Zephyr OmniSense BioHarness-5.0 sensor to acquire ECG [6].

all samples of GSR, HR and BR from a window of time in the past and categorizes the current CS level as either “low”, “moderate”, or “acute”. The real labels used to train the network were obtained with a Fast Motion Sickness Scale (FMS). This procedure was used for detection training. A similar procedure was used for prediction training, categorizing a different sample in the future instead of the sample at the current time. The results produced with only 2 minutes

of previous data achieved accuracies of 97.44% and 87.38% for detection and prediction, respectively. Qu *et al.* [24] proposed a LSTM-Attention Neural Network for real-time CS detection. The subjects were equipped with HTC Vive trackers, 2 in the ankles and one on the torso for tracking the position of the entire body, taking into account data given by the HMD and the controllers hold by both hands. In addition to that, a BIOPACK sensor was used to track ECG (from which various feature such as HR are extracted) and Electrodermal Activity (EDA) data. Each subject participated in a passive and immersive experience in which the scene is moving to induce CS. During the experience, the subject provided feedback on the CS level pressing a button on the controller. The data collected were used to train the LSTM-Attention Model that can be loaded into any VR system. They reached an accuracy of 96.58%. These two studies are just examples of the very high number of state-of-the-art studies that also rely on physiological data to perform CS detection. Even if their results are remarkable, the need for additional sensors that acquire physiological data makes these methods difficult to adapt to a commercial application. Kundu *et al.* [25] opted for an explainable ML approach using three different ML models: explainable boosting machine, decision tree and logistic regression. They trained their models with publicly available physiological and gameplay train datasets for CS. The best results were achieved with the explainable boost machine method, with accuracies of 99.75% and 94.10% for the physiological and gameplay [26] datasets, respectively. The gameplay dataset is comprised of 22 different features from the sources such as candidate profiles, questionnaires, user field of view, user position, and speed of the game in playtime[25]. This study shows that a sufficiently high accuracy can be achieved just with data coming only from a HMD, a more practical approach for most uses. Kundu and Hoque[27] developed a Differentially Private CS Detection. They developed four private neural network structures: long short-term memory, gated recurrent unit, CNN, and multilayer perceptron. They trained this model with two different datasets, Simulation 2021 [28] [29] and Gameplay [30] [26]. The Simulation 2021 dataset contains eye and body tracking data and physiological data while the Gameplay Dataset is the same used in the previous study, with 22 gameplay attributes. With Simulation 2021 and Gameplay, they achieved accuracies of 92% and 91%, respectively. Shimada *et al.* [31] achieved an accuracy of 80% using an attention-based long short-term memory fully convolutional network trained with eye-related indices for high-frequency CS detection. Islam *et al.* [28] proposed a deep fusion network to predict CS from heterogeneous data that is acquired entirely by the sensors available in the HMD. It is a multi-modal network that includes a 3D-CNN model and a Time-distributed CNN-LSTM model. The 3D-CNN model processes the stereoscopic video, the optical flow, and the disparity map to learn the spatiotem-

poral features while the CNN-LSTM model learns the features by processing the eye-tracking and head-tracking data. The data collected in this study comes from 4 simulation games, and it forms the Simulation 2021 dataset [29] that Kundu and Hoque [27] also used. Each sample of the dataset is labeled with either none, low, moderate or acute presence of symptoms of CS. Between all the possible modes Islam *et al.* [28] tested, they achieved the best results combining only eye-tracking and head-tracking data, with an achieved accuracy of 87.7%. The distinction that seems to be more difficult for this net is between low and moderate symptoms of CS. The other modalities (including stereoscopic video, the optical flow, and the disparity map) performed poorly. Choi et al. [32] approach the prediction of CS as a natural language processing problem. They developed a time-large language model that processed HMD sensor data. They compared their model with well-known time series forecasting models and found out that the best results are achieved with PatchTST, with a mean absolute error of 0.931 and a root mean square error of 1.678. Between recent studies in the field of automatic CS detection, studies on methods that also rely on the use of physiological data are far more present than studies on methods that rely only on HMD sensor data. Relying only on HMD data allows the adoption of CS detection techniques to all the application fields of VR, which is crucial for the spread and growth of VR technologies.

Chapter 3

Proposed Method

3.1 The dataset

The dataset comes from a subjective test that involved 60 participants immersed in VR with a HMD. The HMD recorded the information on the six degrees of freedom of each participant's head, and this information was later stored in six images called "heatmaps", one for each degree of freedom. Three position heatmaps are related to translation movements in the three-dimensional physical space, one heatmap for each axis. Each of these heatmaps is 400 pixels wide, while the height varies. Every column of these position heatmap represents a centimeter in the interval between -2m and +2m and every row represents one second of subjective test. To be more specific, the intensity of a pixel corresponds to how much the user remained in that centimeter of space in that second of time. If the pixel is black, it means that the user was not in that centimeter in that specific second. If the pixel is gray, it means that the user was in that centimeter at that specific second of time, and the amount of time the user remained there is indicated by the brightness of the pixel. The other three heatmaps are orientation heatmaps. They track the three possible rotations of the head: yaw, pitch, and roll. An illustration of these rotations is shown in Figure 3.2. Each pixel has the same meaning as in the case of position heatmaps, with the exception of representing degree angles instead of centimeters. This means that each column represents a certain angle related to that type of rotation from the starting position. These heatmaps are 180 pixels wide, each column representing one degree of rotation. For roll and pitch, the columns describe a rotation in the interval between -90° and $+90^\circ$ from the starting position. For yaw, the possible rotation is in the interval between -180° and $+180^\circ$. To fit this interval with only 180 pixels, the absolute value of the rotation is stored, with only the direction of rotation being lost. As a consequence, left and right yaw rotations

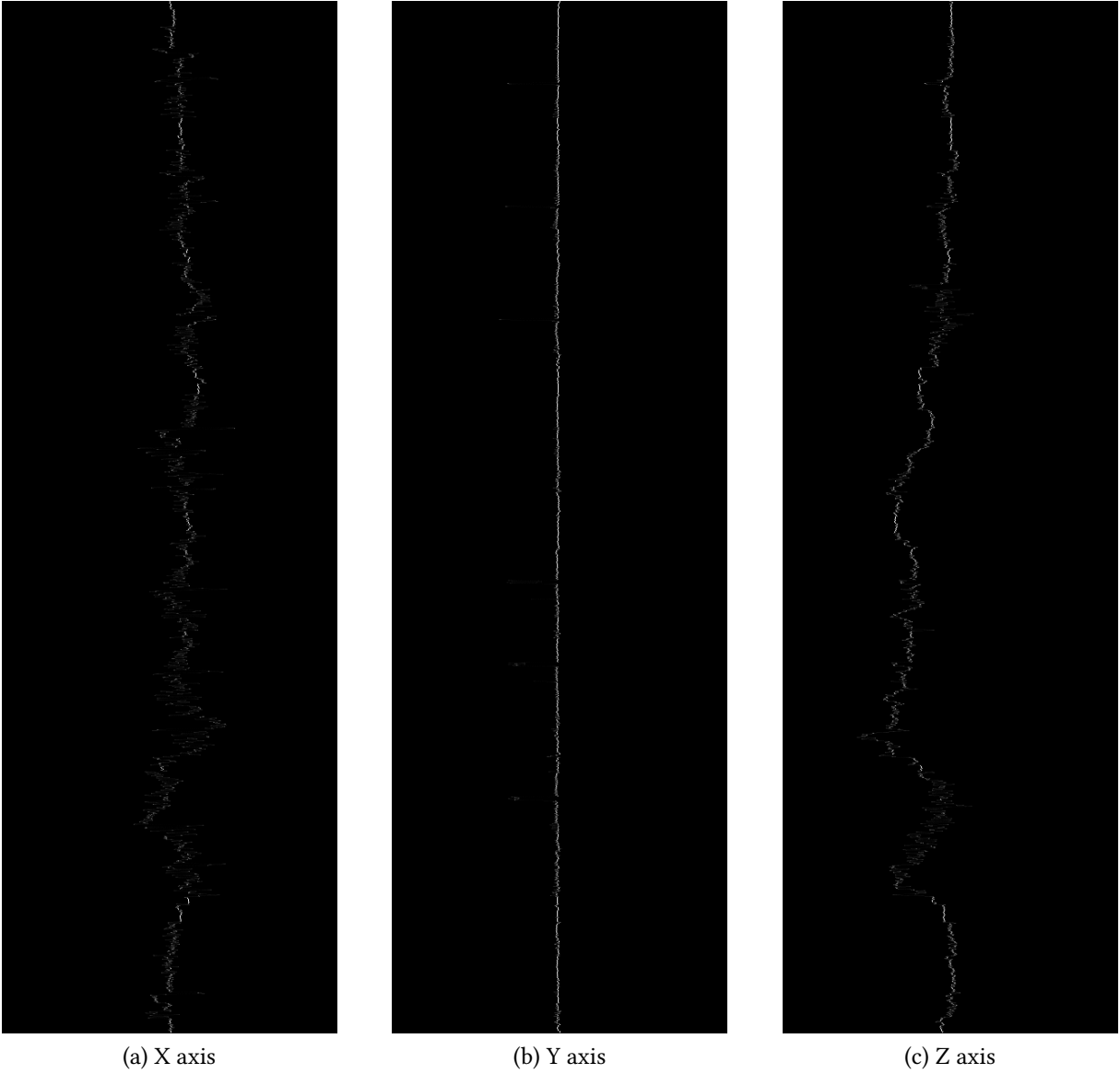


Figure 3.1: Example of position heatmaps belonging to the same subjective test. Each heatmap represent the information regarding translation in a specific axis.

are stored in the same manner, and their heatmaps are indistinguishable. The column in the left margin of a yaw heatmap indicates around 0° , the starting position of the head, and the column in the right margin of the heatmap indicates 180° . An example of a set of position heatmaps is shown in Figure 3.1 and an example of a set of orientation heatmaps is shown in Figure 3.3.

The model used for the detection contains two encoders data taking RGB images as input (further details are provided in Section 3.2). RGB images are colored images with three channels, so the heatmaps cannot be used directly by the encoders in their current form. We decided to merge three related positional heatmaps into a singular RGB image, as shown in Figure 3.4a, and we also merged the three related orientation heatmaps into another RGB image,

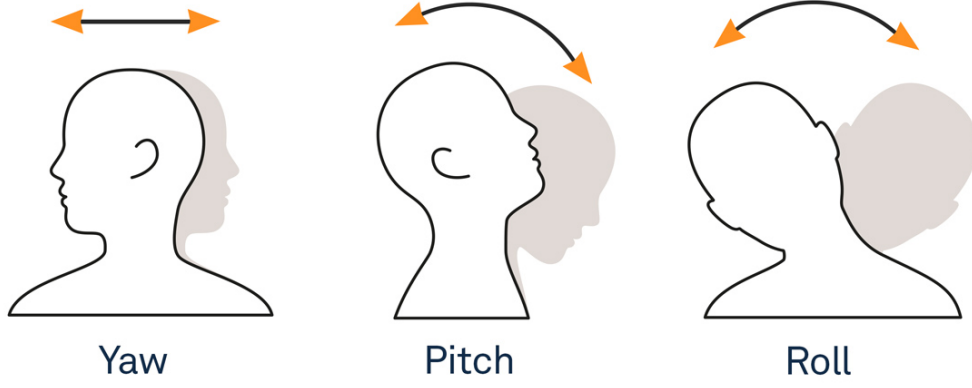
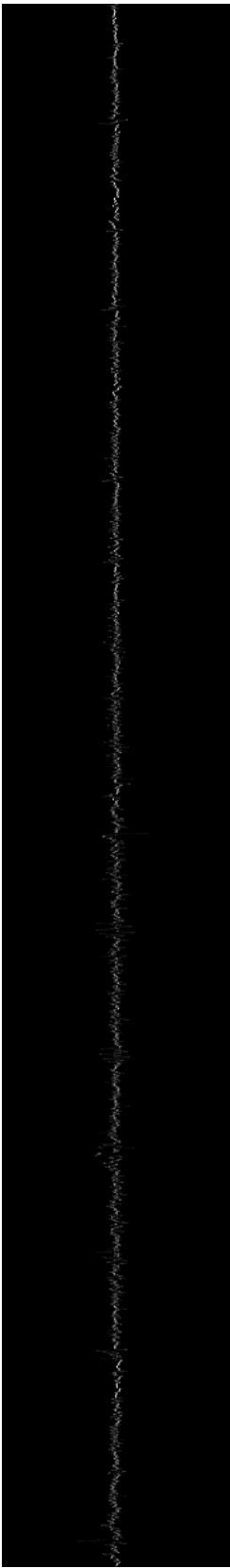


Figure 3.2: Illustration of the three degrees of freedom of rotation of the human head [7].

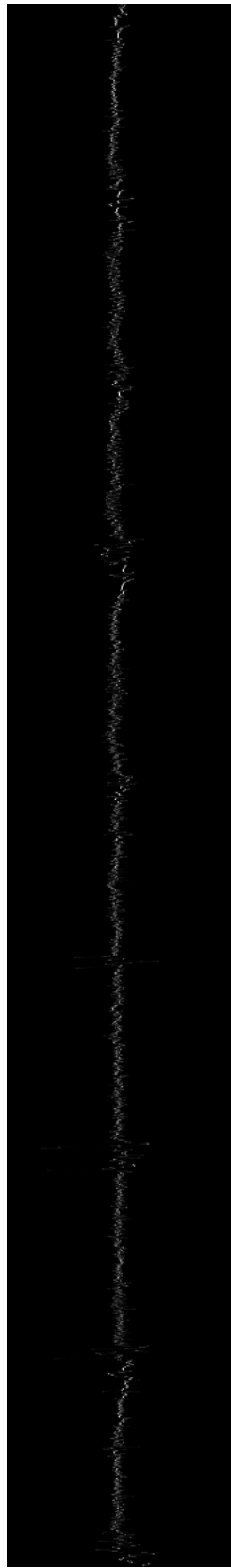
as shown in Figure 3.4b. These two images represent all the information about the six degrees of freedom of a test subject. The heatmaps have also been cut, keeping only the first 600 rows for each heatmap so that all elements of the dataset represent 10 minutes of subjective test. Finally, each heatmap has been normalized. Heatmaps have been represented as tensors, with values in the range from 0 to 1. To normalize the heatmaps, all values in the tensors that store them have been scaled to guarantee that the maximum value inside the tensor is 1. The last step was to pair each set of heatmaps with the corresponding label. The two possible labels for each set of heatmaps are “absent” and “present”, and they refer to either absence or presence of symptoms of CS during the subjective test. Labels have been created from the scores collected from the Simulation Sickness Questionnaire (SSQ) that each participant had to compile during the subjective test. In the SSQ, participants had to assess the intensity of each symptom of CS with a score from 1 to 4. The symptoms are grouped into three sets: Nausea (N), Oculomotor (O), and Disorientation (D). For each set the scores of the corresponding symptoms are added. Finally, the three components are combined to obtain the Total Severity (TS) as shown in Equation (3.1).

$$TS = (N + O + D) \times 3.74 \quad (3.1)$$

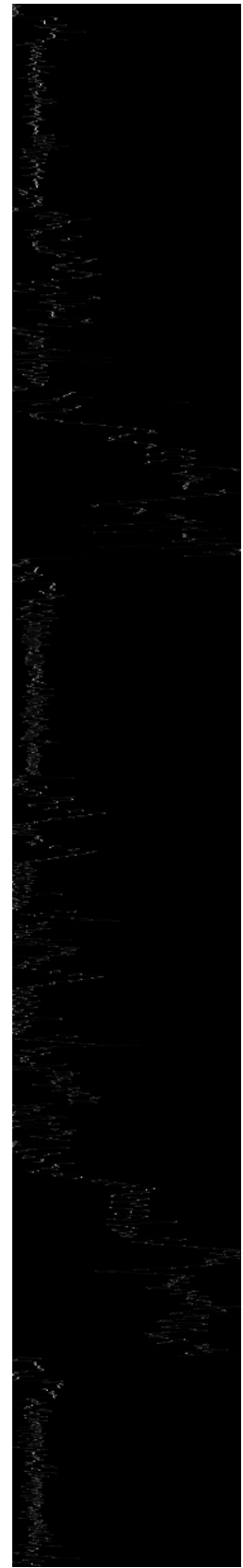
The two final possible classes of the labels are obtained by applying a threshold to the TS: each sample with a TS under 40 is labeled as “absent”, otherwise it is labeled as “present”. In summary, a sample in the dataset contains two cut and normalized RGB heatmaps, a position one and an orientation one, paired with the corresponding label.



(a) Roll

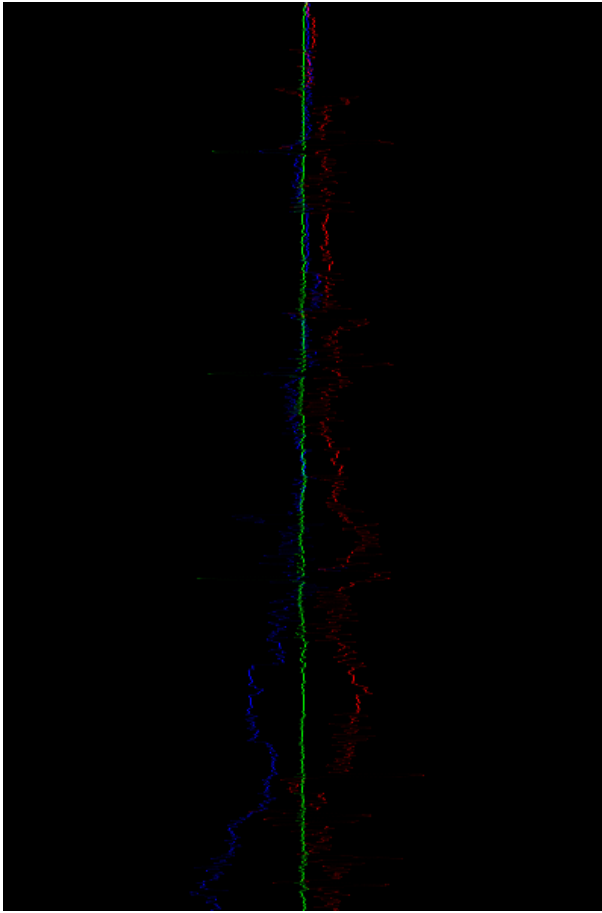


(b) Pitch

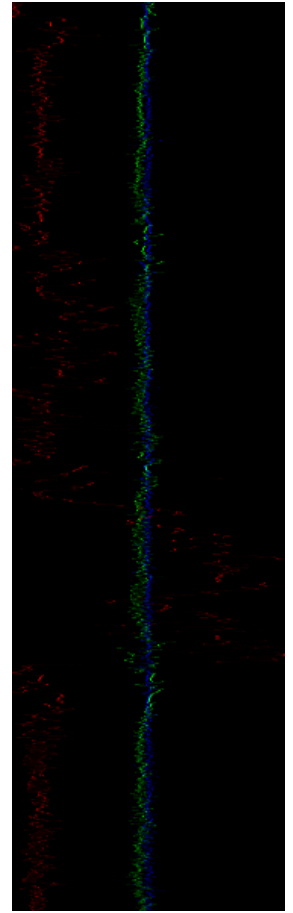


(c) Yaw

Figure 3.3: Example of orientation heatmaps belonging to the same subjective test. Each heatmap represent the information regarding rotation in a specific orientation.



(a) RGB position heatmap



(b) RGB orientation heatmap

Figure 3.4: The six heatmaps merged into two RGB images, cut to 10 minutes and normalized.

3.2 Proposed model

The first element in our model is the encoder. In neural networks, encoders are usually part of a larger structure called an autoencoder. An autoencoder is made up of two components, an encoder and a decoder, and its purpose is to optimally code unlabeled data. The encoder takes some type of data as input, for example, images or audio files, and encodes it to a reduced dimensionality to optimally represent the most important features of the original data. The encoded data is now in the feature space. The decoder performs the opposite task, which consists in taking the encoded data and decoding it back to the space of the original data. Autoencoders are usually trained to regenerate the original data from the encoded data with the highest fidelity, and they are used in various fields, such as anomaly detection or image compression. An illustration of an autoencoder is shown in Figure 3.5. Autoencoders usually belong to the realm of unsupervised learning where data is unlabeled. In our case, the entire dataset is labeled, and our approach is a supervised learning approach, meaning that the labels will be used to evaluate the model training process. The model uses only the encoders to extract the features needed to classify the data, without the need of decoding. The labels produced by the model are compared with the true labels to compute the loss needed for the training process. Encoders are usually implemented with a CNN. A visual representation of a CNN is shown in Figure 3.6. A standard structure for a CNN is characterized by three elements: a set of convolutional layers, a flatten layer, and a set of fully connected layers. Given an image, a convolutional layer applies a kernel over the image to perform convolution, an operation that extracts spatial features from the starting image. The output of the convolution is given to an activation function, a function needed to add non-linearity to the outputs produced by the neurons of the network. After the application of the activation function, a pooling layer

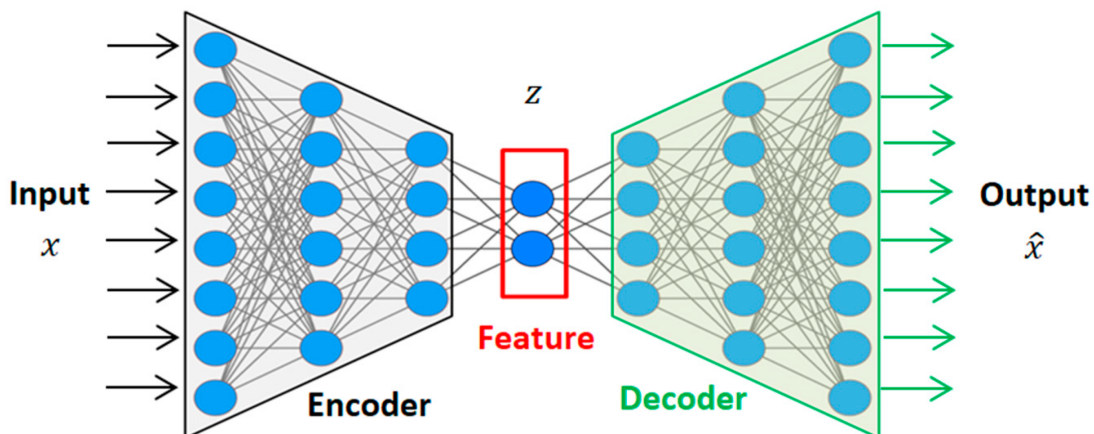


Figure 3.5: Illustration of the autoencoder structure [8].

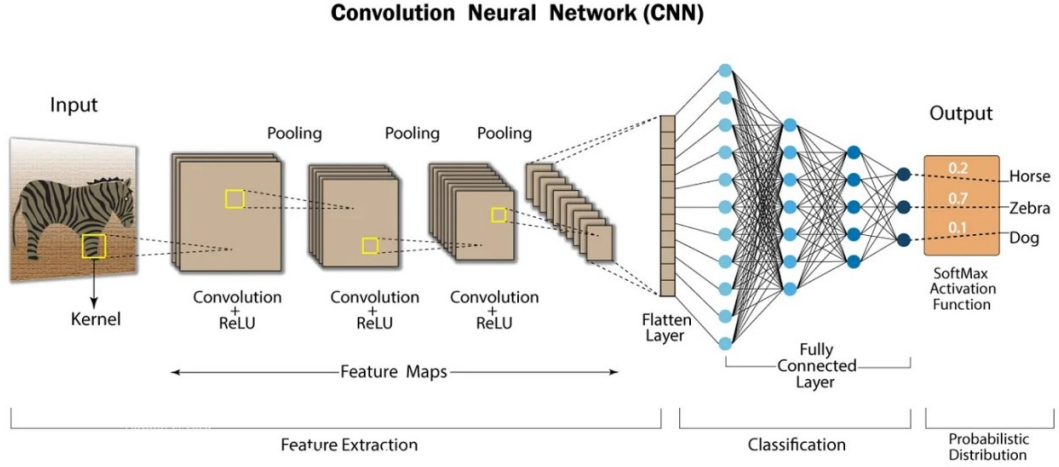


Figure 3.6: Illustration of a CNN. Convolutional layers, followed by a flatten layer and the classification layers [9]

downsamples the data. Downsampling brings numerous benefits: it eliminates redundant information, it increases robustness to small variations and it widens the receptivity of neurons in the next layers [33]. As a result of this process, the output of the pooling layer will be a set of images of a smaller size. In our model, the convolutional portion of the network is composed of three convolutional layers with a 3x3 kernel and a Rectified Linear Unit (ReLU) activation function, as described in Equation (3.2).

$$Relu(z) = \max(0, z). \quad (3.2)$$

The convolution is executed with a stride of 2. This means that the kernel will be shifted two pixels instead of one each time it is applied, consequently halving the dimension of the output. For this reason, our implementation does not include any pooling layer to reduce the dimensions of the output. From the starting image, the first layer produces 8 images. The second and third layers produce twice the number of images that they take as input, so the numbers of images produced by the second and third layers are 16 and 32, respectively. The flatten layer takes the 32 images produced by the convolutional layers and stacks them in a singular dimension. This is necessary to adapt the images to the input structure that the fully connected layers require for their input data. In Figure 3.6 the fully connected layers and a soft max function already produce the classification probabilities for the input image, but our model is more complex since we need two encoders and a classification based on their combined outputs. In the proposed model, the encoder ends with only one fully connected layer that reduces the output of the flatten layer to a desired number of features that represent the original image, and the classification task is executed outside the encoder structure. The

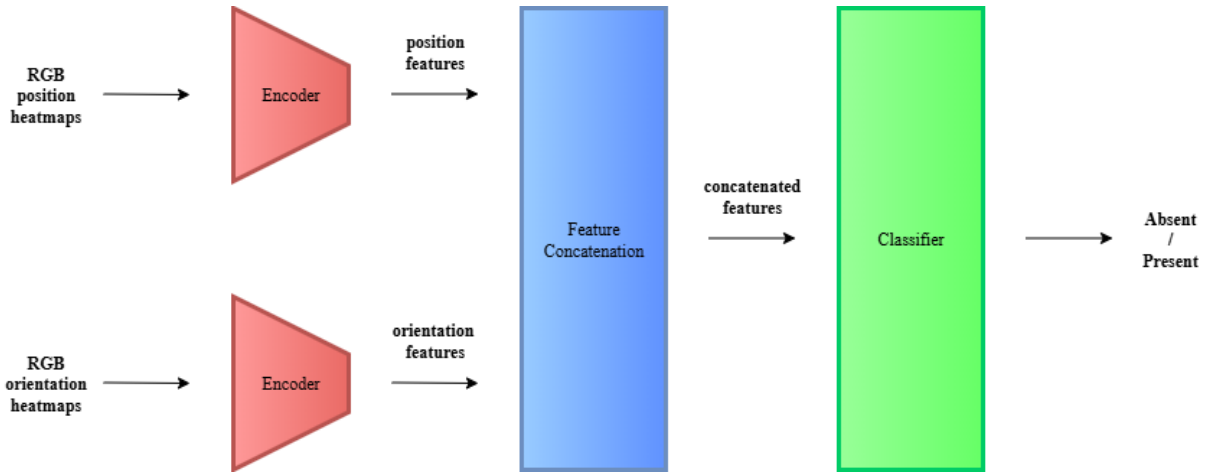


Figure 3.7: Diagram of our model. Position and orientation heatmaps are encoded and the resulting feature are concatenated for classification. The final output will be a label signaling either absence or presence of CS.

structure of our model is shown in Figure 3.7. The two RGB heatmaps are given to two different encoders that will adapt to the input sizes of the related heatmap, extracting the features from the RGB position heatmap and the RGB orientation heatmap separately. The two sets of features are then concatenated into one larger set of features representing the full original sample data that the classifier will use. The classifier is a single fully connected linear layer as those that form the last portion of the CNN structure in Figure 3.6, which outputs a tensor with only two values called “logits”, the likelihood scores for the input heatmaps to belong to the two possible classes (absence or presence of symptoms of CS). The input heatmaps are classified with a label corresponding to the logit with the biggest value. We also wanted to study what a similar structure can achieve with only one type of heatmaps, either position or orientation heatmaps, to better understand the impact of each type of heatmap during the learning process separately. The model used for this task is shown in Figure 3.8. In this case, only one encoder is needed to encode the features of the desired type of heatmaps. The feature concatenation block is not needed since the single encoder produces only one set of features that are given as input directly to the classifier.

3.3 Training and testing the model

The dataset is divided into two parts. The model is trained on 80% of the dataset, and the remaining 20% is designated to test the trained model. The training dataset is further divided to apply the k-fold technique with 5 folds. The K-fold technique is used with small datasets like the one used in this thesis. It consists of splitting the training datasets in k folds, which are

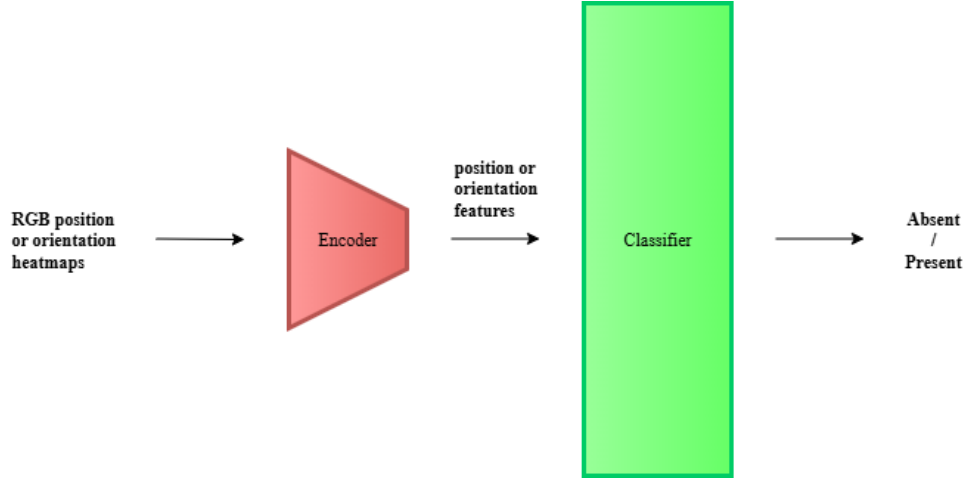


Figure 3.8: Diagram of our model. Position and orientation heatmaps are encoded and the resulting feature are concatenated for classification. The final output will be a label signaling either absence or presence of CS.

subdivisions of the dataset, and the validation set is not the same fold for the entire duration of the training process. One fold validates the training executed on the other folds of the training dataset, and the fold that acts as the validation set changes after a certain number of epochs. With the k-fold split, the net is trained with the entire training dataset, even if not at the same time, and all the folds are designated as the validation set for a certain number of epochs at different stages of the training. A scheduler linearly decreases the learning rate at every epoch, with a starting value of 10^{-4} and decreasing it to half of the starting value. The optimizer used was the Adam optimizer, and the loss function used was the Binary Cross-Entropy loss, described in Equation (3.3), with N representing the batch size, y_i representing the true label of sample i (0 or 1) and p_i representing the probability that the value of y_i is 1.

$$BCE = -\frac{1}{N} \sum_{i=1}^N (y_i \log(p_i) - (1 - y_i) \log(1 - p_i)). \quad (3.3)$$

The objective is to lower the value of the loss as much as possible, because a lower loss indicates that more samples are correctly classified. After the training phase, the generalization performance of the trained model is tested evaluating the labels the model produces on the test set. We used three different metrics: accuracy, recall, and precision. Accuracy is described in Equation (3.4), where TP, TN, FP, FN correspond to the number of true positive, true negative, false positive, and false negative classifications, respectively.

$$accuracy = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (3.4)$$

Accuracy is a metric that describes the overall performance of the model, taking into account all the possible classification possibilities. Recall (also called “true positive rate”) is described in Equation (3.5).

$$recall = \frac{\text{correctly classified positives}}{\text{total actual positives}} = \frac{TP}{TP + FN}. \quad (3.5)$$

Recall is an important metric because it quantifies the ability of the model to correctly classify a positive sample when one is given. In our case, this metric corresponds to the success rate in correctly labeling a sample with presence of CS when the current sample is an actual positive with presence of CS. Precision is described in Equation (3.6).

$$precision = \frac{\text{correctly classified positives}}{\text{total positive classifications}} = \frac{TP}{TP + FP}. \quad (3.6)$$

This metric describes the ratio between the number of true positives and the number of positive classifications. In our case, this ratio describes the percentage of samples classified as affected by CS have actually been sick.

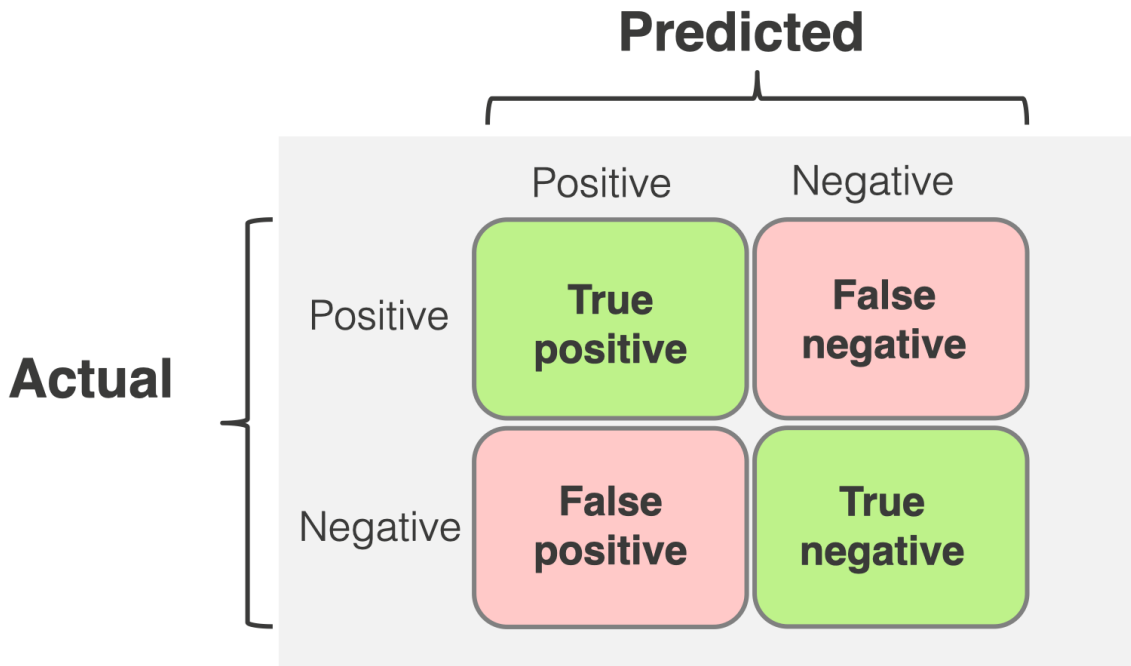


Figure 3.9: An example of confusion matrix. The two rows correspond to the true labels of the samples and the two columns correspond to the labels predicted by the model [10].

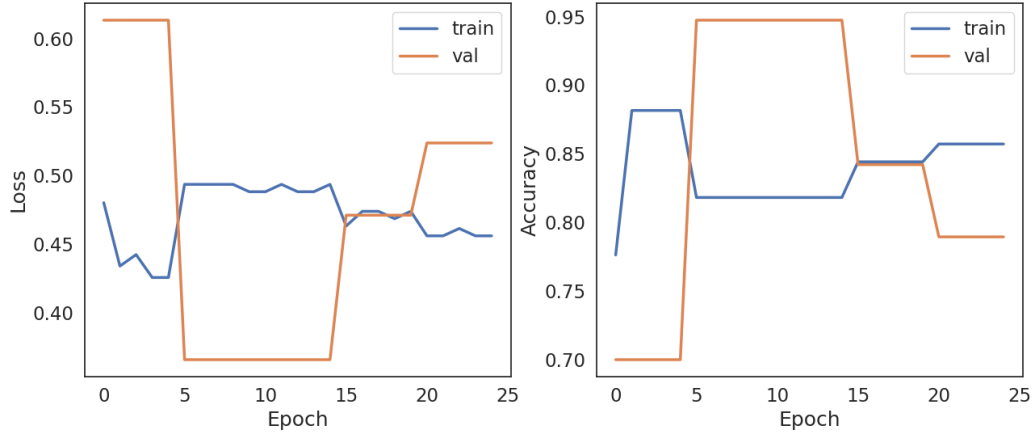
To visualize the relation between the classification produced by the model and the true labels of the samples, confusion matrices will be used multiple times in Chapter 4, where the results of the various tests performed with our model will be discussed. An example of

a Confusion Matrix (CM) is shown in Figure 3.9. In a CM, each row contains all samples characterized by one of the true labels (in our case, the first row contains all samples with “absent” as their true label and the second row contains all samples with “present” as their true label) while each column contains all samples that have been classified with a specific label by the model (in our case, all samples classified as “absent” are in the left column, while all samples classified as “present” are in the right column). Consequently, each section of the CM contains the number of samples with a specific true label and classified with a specific label (for example, the upper left section contains the number of “absent” samples that have been correctly classified with an “absent” label).

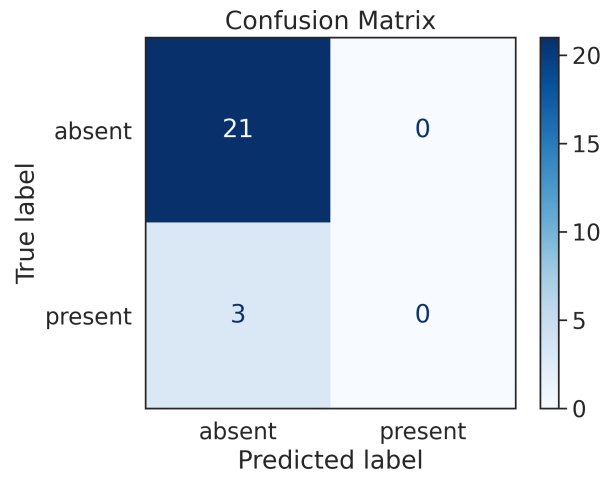
Chapter 4

Experimental Results

The first tests were performed with the original dataset that contained 120 samples from subjective tests (each participant played two different games during the test). Between the 120 samples in the dataset, only 18 samples are labeled as “present” while the other 102 samples are labeled as “absent”. Therefore, “Present” samples represent only 15% of the dataset. Such an unbalanced dataset might cause problems in learning the essential features that distinguish one class from the other. The first training and test results, shown in Figure 4.1, are performed with only the position heatmaps and the one-encoder model shown in Figure 3.8, with the encoder producing a tensor with 192 values representing the features encoded from the heatmaps. The first results clearly demonstrate the negative impact of an unbalanced dataset on the effectiveness of the training. The loss and accuracy score curves that describe the training phase are shown in Figure 4.1a. It is noticeable that both validation curves are flat for a certain number of epochs, then they rapidly change in one specific epoch, and then return to be flat. If we take into account the fact that k-fold is used for the training data, the flat sections clearly identify one particular phase of the k-fold when the validation set remains the same, while the abrupt change happens when a new validation fold is selected for the next k-fold configuration. Observing the curves of training loss scores, we find a similar behavior. This suggests that during each k-fold phase, the model is not learning how to distinguish the two classes in the dataset. This is confirmed by the CM shown in Figure 4.1b where all samples in the test dataset are classified as “absent”. Therefore, the model learned to recognize the features of absent data but it did not learn how to distinguish them from the few “present” samples it was given during training. Figure 4.1b also shows that the test dataset is composed of 21 samples labeled as “absent” and 3 samples labeled as “present”, reflecting once again how deeply unbalanced the dataset is.



(a) Curves for loss and accuracy scores in training.

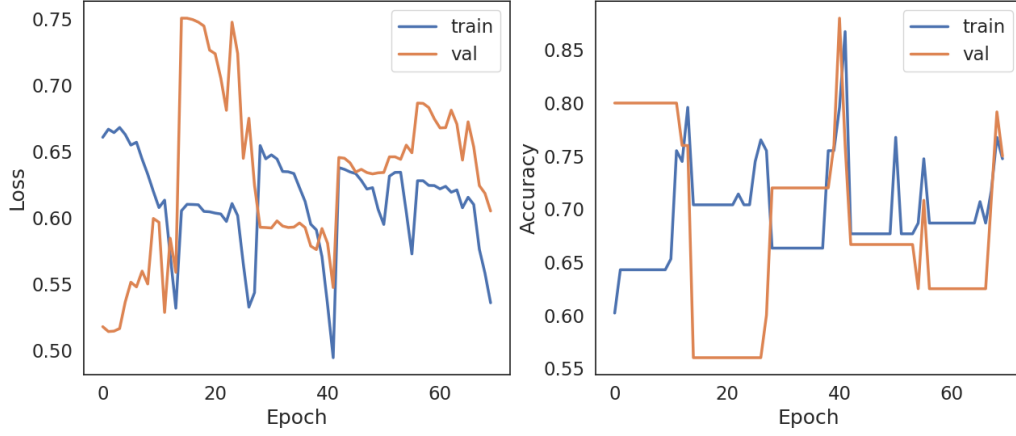


(b) CM obtained with Test results.

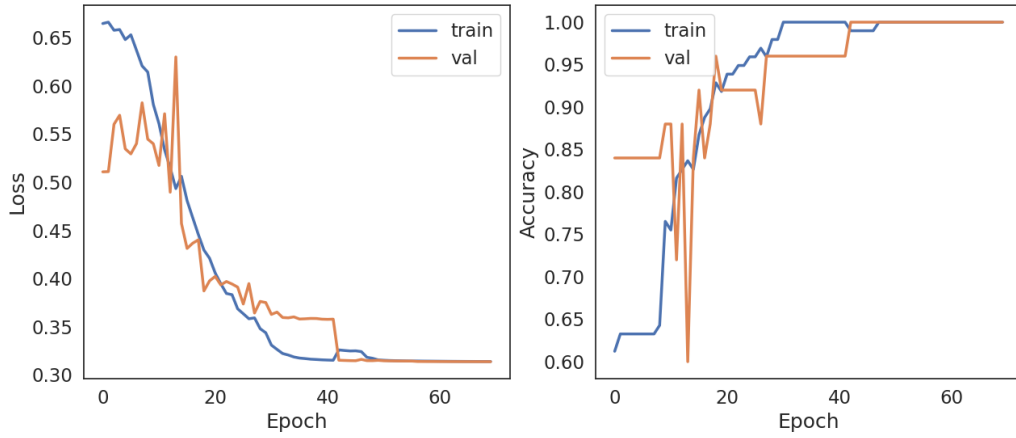
Figure 4.1: Results of training and testing the one-encoder model with only position heatmaps from the original dataset.

The model has an accuracy of 87.5%. If taken by itself, the accuracy would suggest that the model performs well. This highlights the importance of using multiple metrics. Indeed, the recall and precision scores are both 0% since no “present” sample is detected, representing the category of true positives in Equations (3.5) and (3.6). These considerations highlight that the model needed more “present” samples while training.

The proposed solution was to apply data augmentation only on the “present” samples in the dataset so that it would be more balanced. Given the symmetries of the heatmaps, each heatmap was duplicated by flipping it in the horizontal dimension, without any modification to the vertical dimension. It conceptually means creating new samples, with every rotation and every translation of the participant being mirrored, without changing the temporal evolution of the samples. We then created multiple copies of both normal and mirrored heatmaps to reach two different percentages of “present” samples in the dataset: 33% and 50%. We tested



(a) Curves for loss and accuracy scores obtained training only with position heatmaps.



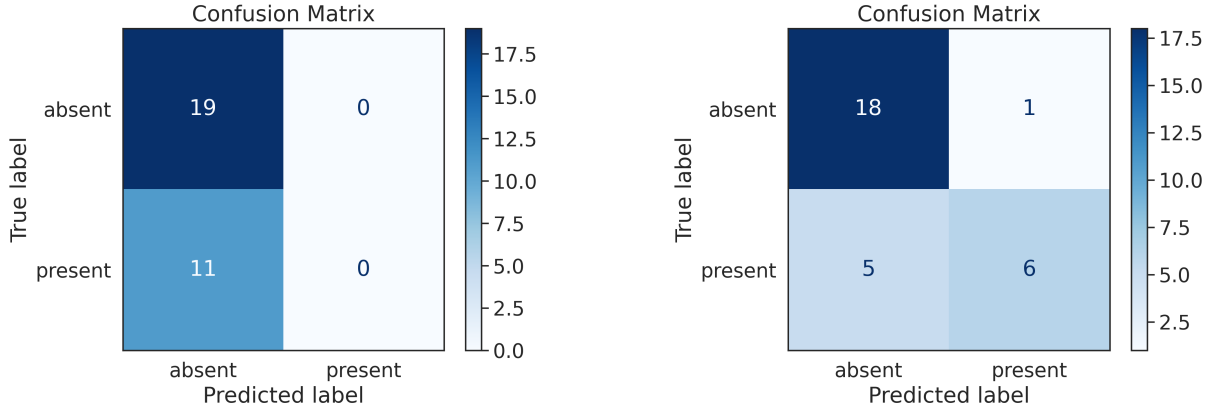
(b) Curves for loss and accuracy scores obtained training only with orientation heatmaps.

Figure 4.2: Results of training the one-encoder model with a dataset composed of 33% “present” samples.

these new augmented datasets with the one-encoder model, first with position heatmaps and then with orientation heatmaps separately.

The results of training the model with a dataset of 33% “present” samples are shown in Figures 4.2 and 4.3. In Figure 4.2a we can see the loss and accuracy curves in the training phase using only position heatmaps. Although they are not completely flat as in the previous case, the validation accuracy curve suggests that the unbalance problem has not been solved yet since it follows the same behavior as the curves in Figure 4.1a. This is confirmed by the CM obtained while testing, shown in Figure 4.3a. All the samples in the test set have been classified as “absent” once again. The accuracy score drops to 63.3% while the recall and precision scores remain at 0%. With only position heatmaps and 33% “present” samples, the model was still unable to detect these particular samples.

The use of orientation heatmaps led to better results. The training loss and accuracy curves,



(a) CM obtained testing the model trained with only position heatmaps.

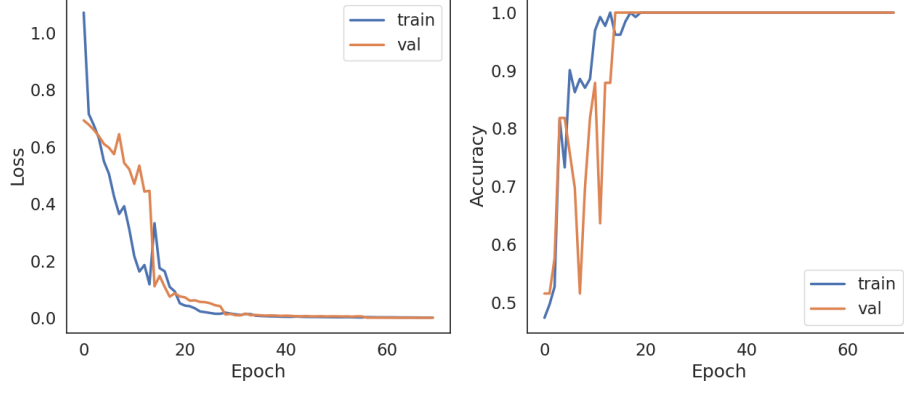
(b) CM obtained testing the model trained with only orientation heatmaps.

Figure 4.3: Results of testing the one-encoder model trained with a dataset composed of 33% “present” samples.

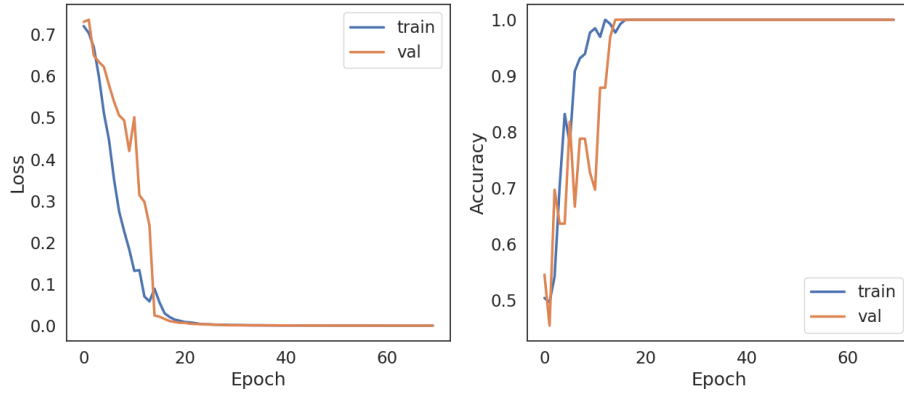
shown in Figure 4.2b, show that the model appears to be learning, since the losses of both training and validation significantly decrease as the epochs pass and the accuracy curves increase, reaching high scores from around epoch 30 onward. This is confirmed by the CM shown in Figure 4.3b. The model achieved an accuracy of 80%, a recall of 54.5% and a precision of 85.7%. The recall score highlights how the model is now recognizing about half of the “present” samples correctly, and the high precision score indicates that the model has not learned to classify “absent” samples incorrectly (just one sample between 19 “absent” samples). These results seem promising, since the model has now partially learned how to encode the features of “present” samples and classify them correctly, but not yet satisfactory.

The results obtained with the dataset composed of 50% “present” samples show much more improvement. The curves describing the training with only position heatmaps are shown in Figure 4.4a. The model completes the learning process in around 20 epochs. The final results of the test are shown in Figure 4.5a. The model achieved an accuracy of 97.5%, a recall of 100%, and a precision of 95%. This time the model was able to recognize all the “present” samples and classifies only one sample incorrectly over the 21 “absent” samples in the test set. This indicates that the model has finally learned how to encode and correctly classify the “present” samples without losing the ability to distinguish them from the “absent” samples. The model obtains similar results if trained only with orientation heatmaps from the same dataset, as shown in Figures 4.4b and 4.5b. With an accuracy of 95%, a recall of 100%, and a precision of 90.9%, the model correctly classifies all “present” samples while incorrectly classifying only two samples over the 20 “absent” samples in the test set.

All training and testing procedures discussed were performed with tensors of a size of 192



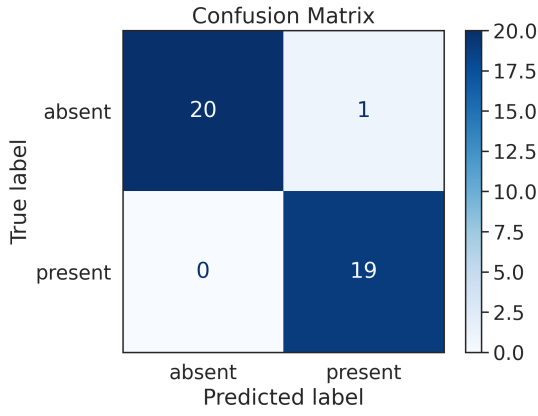
(a) Curves for loss and accuracy scores obtained training only with position heatmaps.



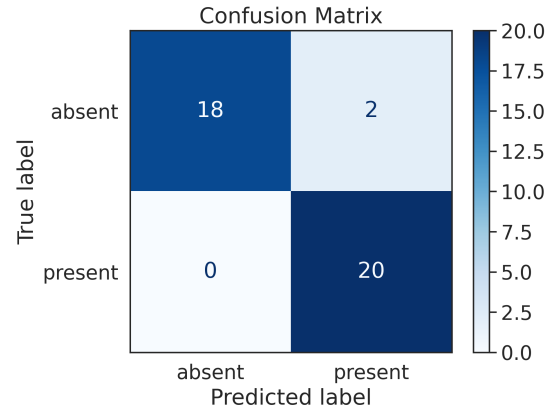
(b) Curves for loss and accuracy scores obtained training only with orientation heatmaps.

Figure 4.4: Results of training the one-encoder model with a dataset composed of 50% “present” samples.

to store the encoded features, and a starting learning rate of 10^{-4} . Changing these hyperparameters proved to have no positive impact on the model training procedure. The results clearly indicate that the use of a balanced dataset, composed of 50% by “present” samples and 50% by “absent” samples, leads to a drastically better learning process for the model. Consequently, the same dataset was used to train the larger model with two encoders shown in Figure 3.7. We kept 192 as the size of the tensor containing the features extracted by the encoders. As a consequence, the concatenated set that contains features extracted from both position and orientation heatmaps is contained in a tensor of size 384. The training and test results are shown in Figure 4.6. The training accuracies in Figure 4.6a reflect the same behavior as in the case of the one-encoder model, that requires less than 20 epochs to learn from the input heatmaps. The CM in Figure 4.6b shows that the model incorrectly classified 3 “absent” samples, consequently lowering the accuracy to 92.5% and the precision to 86.4%, while the recall remains 100% since the model correctly classifies all “present” samples. Since it is given full information, the model was expected to perform at least as well as in the previous cases;



(a) CM obtained testing the model trained with only position heatmaps.

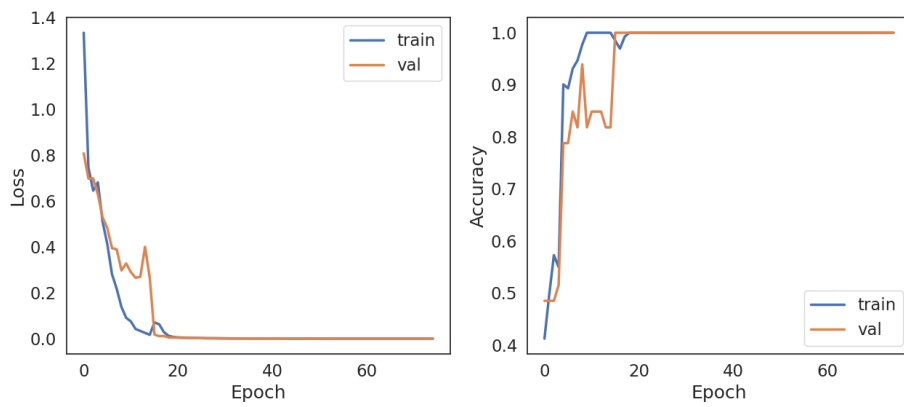


(b) CM obtained testing the model trained with only orientation heatmaps.

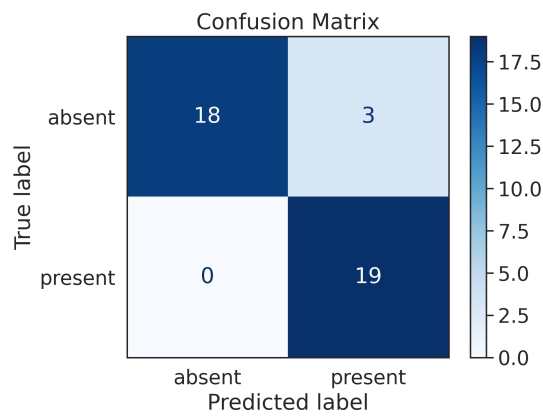
Figure 4.5: Results of testing the one-encoder model trained with a dataset composed of 50% “present” samples.

instead, it obtained worse results.

Various hyper-parameters have been modified and better results were obtained by setting the size of output features of each encoder to 5, resulting in a concatenated set of size 10. The results with the applied change are shown in Figure 4.7. In this case, the accuracy curves in Figure 4.7a stabilize after almost 40 epochs, reflecting the fact that it takes longer to train this version of the model. This could be explained by the fact that, disposing of only 10 total features to represent the data instead of the previous set of 384 features, the model requires more epochs to optimally adjust the weights. On the other hand, the CM in Figure 4.7b reports similar results to the case of the one encoder model in Figure 4.5b. The accuracy achieved is 95%, the recall achieved is 100% and the precision achieved is 90.5%. This was the best result obtained with the larger model since the manual tuning of other hyper-parameter did not yield better results.

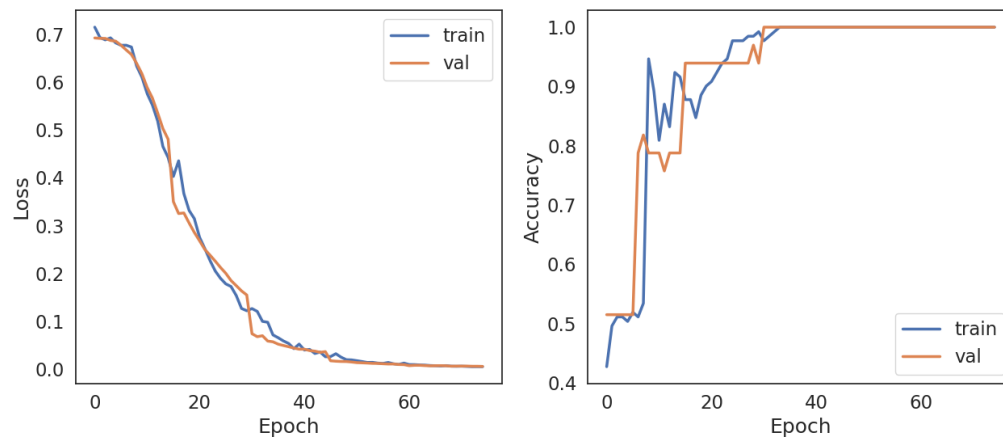


(a) Curves for loss and accuracy scores in training.

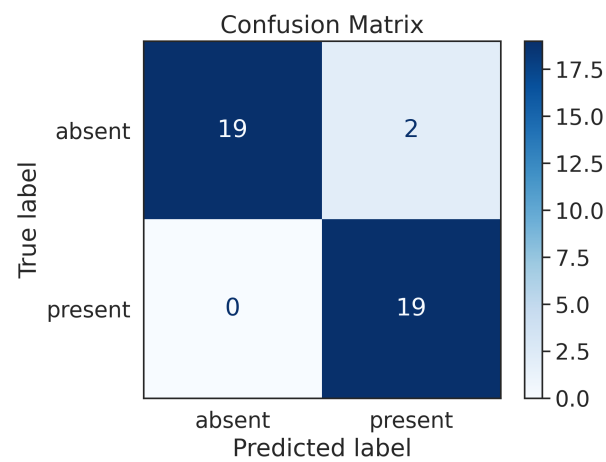


(b) CM obtained with Test results.

Figure 4.6: Results of training and testing the larger model with the number of features from each encoder set to 192.



(a) Curves for loss and accuracy scores in training.



(b) CM obtained with Test results.

Figure 4.7: Results of training and testing the larger model with the number of features from each encoder set to 5.

Chapter 5

Conclusions

The previous chapters in this thesis depicted an overview of the CS detection topic and describe in detail the model we developed and the dataset on which the training is based. In the first place, this thesis discussed the main aspects of the related literature, from causes and contributing factors to state-of-the-art techniques to detect and reduce symptoms CS during VR immersion. Then it presents a description of the dataset, introducing heatmaps and the modifications applied to prepare and adapt the dataset to the model requirements, from the raw heatmaps to the cut and normalized RGB heatmaps. It also describes the structure of the model, starting from the description of the main components and further analyzing the layers of neural network that compose them. At last, this thesis showcases the results the model obtained during the numerous training and testing trials, also describing the different adjustments we adopted to improve the performance of the model. To summarize the most important results discussed in Chapter 4, the best performance was obtained with data augmentation and duplication (with 50% “present” samples). The one-encoder model produced the best performance if trained only with position maps, achieving an accuracy of 97.5%, a recall of 100%, and a precision of 95%, classifying incorrectly just one “absent” sample over the 40 samples of the test set. On the other hand, the larger model, trained with both position and orientation heatmaps, achieved an accuracy of 95%, a recall of 100% and a precision of 90.5%, incorrectly classifying two “absent” sample over the 40 samples of the test set. It appears that a model disposing of just one encoder and only one set of heatmaps, either the position or the orientation ones, produces results of sufficiently high quality. The fact that the smaller model can be used with performance comparable to or even better than the ones of the larger model brings an additional series of upsides. For example, a smaller model occupies less space in the memory of anHMD that adopts it to perform CS detection, and it also needs half of

the heatmaps to perform its task. This thesis demonstrates how position and orientation data acquired from an HMD have the potential to be solid assets for CS detection, and the proposed model opens to future directions. For example, if the model could classify heatmaps with variable height size, it could be applicable to real-time scenarios to detect CS as it occurs. The data augmentation applied to the “present” samples can also be improved. “Present” samples in the test set had a high probability of being a copy, or a mirrored version at best, of a sample that the model had already encountered during training. Further research could be conducted on the generalization power of the model regarding the “present” samples. Future research could be conducted to train the model avoiding this critical aspect, for example training with a bigger dataset or with richer data augmentation techniques that modify the original samples in different ways, without altering the temporal relations in the heatmaps. Overall, the findings of this thesis have the potential to be important preliminary step towards efficient CS detection. Finally, as the dataset was obtained with games characterized by different paces, from the slow and relaxed experience of cooking a meal to the frenetic action of a shooting game, the obtained results indicate that the proposed model could be applicable to a wide variety of VR applications to preserve the well-being of the users.

Acronyms

VR Virtual Reality

CS Cybersickness

HMD Head Mounted Display

FOV Field of View

FMS Fast Motion Sickness Scale

GSR Galvanic Skin Response

HR Heart Rate

BR Breath Rate

ECG Electrocardiogram

ML Machine Learning

DL Deep Learning

GVS Galvanic Vestibular Stimulation

EDA Electrodermal Activity

BMI Body Mass Index

RGB Red, Green and Blue Images

CNN Convolutional Neural Network

ReLU Rectified Linear Unit

CM Confusion Matrix

SSQ Simulation Sickness Questionnaire

TS Total Severity

Bibliography

- [1] J. J. LaViola, "A discussion of cybersickness in virtual environments," *ACM SIGCHI Bulletin*, vol. 32, no. 1, pp. 47–56, 2000.
- [2] C. Groth, J.-P. Tauscher, N. Heesen, S. Castillo, and M. Magnor, "Visual techniques to reduce cybersickness in virtual reality," in *2021 IEEE conference on virtual reality and 3D user interfaces abstracts and workshops (VRW)*. IEEE, 2021, pp. 486–487.
- [3] C. Groth, M. Magnor, S. Grogorick, M. Eisemann, and P. Didyk, "Cybersickness reduction via gaze-contingent image deformation," *ACM Trans. Graph.*, vol. 43, no. 4, Jul. 2024. [Online]. Available: <https://doi.org/10.1145/3658138>
- [4] Y. Farmani and R. J. Teather, "Evaluating discrete viewpoint control to reduce cybersickness in virtual reality," *Virtual Reality*, vol. 24, no. 4, pp. 645–664, 2020.
- [5] P. Budhiraja, M. R. Miller, A. K. Modi, and D. Forsyth, "Rotation blurring: use of artificial blurring to reduce cybersickness in virtual reality first person shooters," *arXiv preprint arXiv:1710.02599*, 2017.
- [6] R. Islam, Y. Lee, M. Jaloli, I. Muhammad, D. Zhu, P. Rad, Y. Huang, and J. Quarles, "Automatic detection and prediction of cybersickness severity using deep neural networks from user's physiological signals," in *2020 IEEE international symposium on mixed and augmented reality (ISMAR)*. IEEE, 2020, pp. 400–411.
- [7] "Spatial audio part 3: Building repeatable evaluation systems," available online - last access 21-09-2025. [Online]. Available: <https://www.ceva-ip.cn/blog/evaluating-spatial-audio-part-3-creating-a-repeatable-system-to-evaluate-spatial-audio>
- [8] K. A. Alaghbari, H.-S. Lim, M. H. M. Saad, and Y. S. Yong, "Deep autoencoder-based integrated model for anomaly detection and efficient feature extraction in iot networks," *IoT*, vol. 4, no. 3, pp. 345–365, 2023.

- [9] “What is convolutional neural network — cnn (deep learning),” available online - last access 25-09-2025. [Online]. Available: <https://www.linkedin.com/pulse/what-convolutional-neural-network-cnn-deep-learning-nafiz-shahriar/>
- [10] “How to interpret a confusion matrix for a machine learning model,” available online - last access 30-09-2025. [Online]. Available: <https://www.evidentlyai.com/classification-metrics/confusion-matrix>
- [11] P. Bockelman and D. Lingum, *Factors of Cybersickness*. Springer International Publishing, 2017, p. 3–8. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-58753-0_1
- [12] A. T. Dilanchian, R. Andringa, and W. R. Boot, “A pilot study exploring age differences in presence, workload, and cybersickness in the experience of immersive virtual reality environments,” *Frontiers in Virtual Reality*, vol. 2, p. 736793, 2021.
- [13] K. Petri, K. Feuerstein, S. Folster, F. Bariszlovich, and K. Witte, “Effects of age, gender, familiarity with the content, and exposure time on cybersickness in immersive head-mounted display based virtual reality.” *American Journal of Biomedical Sciences*, vol. 12, no. 2, 2020.
- [14] A. N. Ramaseri Chandra, F. El Jamiy, and H. Reza, “A systematic survey on cybersickness in virtual environments,” *Computers*, vol. 11, no. 4, p. 51, Mar. 2022. [Online]. Available: <http://dx.doi.org/10.3390/computers11040051>
- [15] J. T. Reason and J. J. Brand, *Motion sickness*. Academic press, 1975.
- [16] M. Treisman, “Motion sickness: an evolutionary hypothesis,” *Science*, vol. 197, no. 4302, pp. 493–495, 1977.
- [17] K. Money, “Motion sickness and evolution,” *Motion and space sickness(A 93-55929 24-52)*. Boca Raton, FL, CRC Press, Inc., 1990,, pp. 1–7, 1990.
- [18] G. E. Riccio and T. A. Stoffregen, “An ecological theory of motion sickness and postural instability,” *Ecological psychology*, vol. 3, no. 3, pp. 195–240, 1991.
- [19] S. Weech, T. Wall, and M. Barnett-Cowan, “Reduction of cybersickness during and immediately following noisy galvanic vestibular stimulation,” *Experimental Brain Research*, vol. 238, no. 2, p. 427–437, Jan. 2020. [Online]. Available: <http://dx.doi.org/10.1007/s00221-019-05718-5>

- [20] A. P. B. d. A. Pires, T. R. Silva, M. S. Torres, M. L. Diniz, M. C. Tavares, and D. U. Gonçalves, “Galvanic vestibular stimulation and its applications: a systematic review,” *Brazilian Journal of Otorhinolaryngology*, vol. 88, p. S202–S211, Nov. 2022. [Online]. Available: <http://dx.doi.org/10.1016/j.bjorl.2022.05.010>
- [21] A. Benelli, F. Neri, A. Cinti, P. Pasqualetti, S. M. Romanella, A. Giannotta, D. De Monte, M. Mandalà, C. Smeralda, D. Prattichizzo, E. Santarnecchi, and S. Rossi, “Frequency-dependent reduction of cybersickness in virtual reality by transcranial oscillatory stimulation of the vestibular cortex,” *Neurotherapeutics*, vol. 20, no. 6, p. 1796–1807, Oct. 2023. [Online]. Available: <http://dx.doi.org/10.1007/s13311-023-01437-6>
- [22] “vestibular cortex,” available online - last access 10-09-2025. [Online]. Available: https://en.wikipedia.org/wiki/Vestibular_cortex
- [23] A. S. Fernandes and S. K. Feiner, “Combating vr sickness through subtle dynamic field-of-view modification,” in *2016 IEEE symposium on 3D user interfaces (3DUI)*. IEEE, 2016, pp. 201–210.
- [24] C. Qu, X. Che, S. Ma, and S. Zhu, “Bio-physiological-signals-based vr cybersickness detection,” *CCF Transactions on Pervasive Computing and Interaction*, vol. 4, no. 3, pp. 268–284, 2022.
- [25] R. K. Kundu, R. Islam, P. Calyam, and K. A. Hoque, “Truvr: Trustworthy cybersickness detection using explainable machine learning,” in *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 2022, pp. 777–786.
- [26] “Gameplay dataset,” available online - last access 17-09-2025. [Online]. Available: <https://github.com/tmp1986/UFFCSDData>
- [27] R. K. Kundu and K. A. Hoque, “Preserving personal space: Differentially private cybersickness detection in immersive virtual reality environments,” in *2024 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 2024, pp. 11–20.
- [28] R. Islam, K. Desai, and J. Quarles, “Cybersickness prediction from integrated hmd’s sensors: A multimodal deep fusion approach using eye-tracking and head-tracking data,” in *2021 IEEE international symposium on mixed and augmented reality (ISMAR)*. IEEE, 2021, pp. 31–40.

- [29] “Simulation 2021 dataset,” available online - last access 17-09-2025. [Online]. Available: <https://github.com/tmp1986/UFFCSData>
- [30] T. Porcino, E. O. Rodrigues, F. Bernardini, D. Trevisan, and E. Clua, “Identifying cybersickness causes in virtual reality games using symbolic machine learning algorithms,” *Entertainment Computing*, vol. 41, p. 100473, 2022.
- [31] S. Shimada, P. Pannattee, Y. Ikei, N. Nishiuchi, and V. Yem, “High-frequency cybersickness prediction using deep learning techniques with eye-related indices,” *IEEE Access*, vol. 11, pp. 95 825–95 839, 2023.
- [32] Y. Choi, D. Jeong, B. Kim, and K. Han, “Early prediction of cybersickness in virtual reality using a large language model for multimodal time series data,” in *Companion of the 2024 on ACM International Joint Conference on Pervasive and Ubiquitous Computing*, 2024, pp. 25–29.
- [33] “Wikipedia page on pooling layers,” available online - last access 30-09-2025. [Online]. Available: https://en.wikipedia.org/wiki/Pooling_layer